

Telemetry system for E-Racing Bergamo

Simone Manenti 1048788
Cattaneo Federico 1053265

May 6, 2024

Introduzione

Il progetto

Il progetto prevede la realizzazione di un'applicazione Java che permetta di ricevere, visualizzare e archiviare i dati di telemetria provenienti dalla vettura ERB-01, una monoposto da competizione sportiva progettata dal team E-Racing Bergamo.

Descrizione generale

Prospettive del prodotto:

Il prodotto è progettato per essere utilizzato su un comune PC dotato di Windows e Java 21.

Non ci sono particolari esigenze per quanto riguarda la potenza di calcolo richiesta per la sua esecuzione.

L'archivio nel database interno richiede almeno 100MB di spazio su disco.

Il sistema prevede l'interfacciamento con una periferica esterna, chiamata semplicemente ricevitore, tramite porta USB.

Il ricevitore è un dispositivo sviluppato separatamente dal team E-Racing Bergamo, le cui specifiche sono note.

Funzionalità del prodotto

- Connettersi e ricevere correttamente i dati provenienti dal ricevitore.
- Archiviare i dati ricevuti all'interno del database locale.
- Visualizzare i dati in modo intuitivo tramite interfaccia grafica.
- Esportare i dati su file .csv per archiviazione o per ulteriori analisi

Caratteristiche dell'utente:

L'applicazione è rivolta esclusivamente al personale del team E-Racing Bergamo, in particolare il sistema è sviluppato appositamente come strumento di supporto per il dipartimento di "Controllo" che si occupa della programmazione e dello sviluppo dei sistemi elettronici della monoposto.

Si prevede quindi che il personale sia pienamente a conoscenza del dominio al quale l'applicazione appartiene.

Vincoli

Tutte le funzionalità offerte dall'applicazione sono a disposizione dell'utente senza la necessità di autenticarsi.

Benché sia possibile collegare fisicamente più unità riceventi allo stesso PC, l'applicazione consente un unico canale di comunicazione per volta.

Specifica dei requisiti

Introduzione

Il progetto prevede la realizzazione di un'applicazione Java che permetta di ricevere, visualizzare e archiviare i dati di telemetria provenienti dalla vettura ERB-01, una monoposto da competizione sportiva progettata dal team E-Racing Bergamo.

Scopo

Il documento esplicita i requisiti del sistema di raccolta dati

3.0.1 Obiettivo

L'obiettivo del prodotto è quello di velocizzare le operazioni di download dei dati telemetrici, evitando il collegamento via cavo ad auto ferma, e di permettere l'accesso ai dati quando il mezzo è in movimento.

3.0.2 Definizioni, acronimi, abbreviazioni

- **TD:** Telemetry Data indica il frame contenente le informazioni sullo stato del mezzo.
- **TX:** Indica l'unità hardware a bordo del mezzo responsabile dell'invio dei dati via wireless verso l'unità ricevente.
- **RX:** Indica l'unità hardware, collegata al PC su cui è in esecuzione il software di telemetria, responsabile della ricezione dei dati.
- **COM:** Porta seriale attraverso il quale avviene lo scambio dati tra l'applicazione e l'unità ricevente.

Requisiti di interfacce esterne:

Lo scopo dell'applicazione è quello di fornire una panoramica generale sullo stato della monoposto di conseguenza l'interfaccia grafica deve riflettere la struttura di un "pannello di controllo" virtuale: la totalità delle informazioni e delle funzionalità devono essere accessibili in un'unica schermata in modo compatto e facilmente interpretabile.

Requisiti funzionali

- Connessione al ricevitore: La selezione del ricevitore avviene tramite menu a discesa, l'apertura e la chiusura della comunicazione si ottiene attraverso un pulsante.
- Archiviazione dati: Il salvataggio dei dati deve essere eseguito in modo automatico durante tutto l'arco temporale in cui la comunicazione è aperta.

Requisiti di prestazioni

- Il sistema supporta un singolo accesso simultaneo ad un'unità ricevente.
- Il database deve archiviare fino a 100 000 registrazioni di TD.
- Il sistema deve visualizzare i dati telemetrici real-time entro 1000ms dalla loro trasmissione da parte dell'unità trasmittente.
- Il sistema deve essere in grado di gestire una frequenza di almeno 2 TD al secondo.

Vincoli di design

L'interfacciamento con l'unità ricevente è vincolante per il funzionamento dell'applicazione.

Requisiti di robustezza e affidabilità

Un malfunzionamento del software non costituisce fattore di rischio per cose o persone per cui non ci sono requisiti minimi di robustezza e affidabilità.

Ingegneria dei Requisiti

Il processo di sviluppo agile ci ha consentito di non focalizzarci eccessivamente nel tentativo di stabilire i requisiti nella fase iniziale del progetto, consapevoli del fatto che questi si sarebbero palesati in corso d'opera.

I requisiti, infatti, si sono evoluti insieme al progetto.

Le tecniche utilizzate per la determinazione dei requisiti sono state:

- **Brainstorming:** Utilizzato principalmente nella fase iniziale, ci ha fornito un insieme di idee che ci ha permesso di inquadrare una prima direzione di sviluppo.
- **Analisi di scenari:** Attraverso scenari ipotetici di utilizzo abbiamo cercato di identificare le caratteristiche chiave per il nostro prodotto.
- **Prototipazione:** La prototipazione ci ha consentito, fin dalle fasi iniziali, di comprendere se i requisiti iniziali fossero nella portata delle nostre capacità e il se progetto fosse effettivamente realizzabile.
- **Dialogo:** Il dialogo con i membri del team è stato fondamentale durante tutto l'arco di sviluppo del software.

I requisiti sono stati classificati attraverso la tecnica MoSCoW.

4.0.1 Must Have

- Gestione porta COM comunicazione con ricevitore.
- Gestione database locale per archiviazione/consultazione storico dati.
- GUI: Visualizzazione real-time dei dati ricevuti.
- GUI: Gestione porta COM.
- GUI: Visualizzazione tabellare storico dati.
- GUI: Visualizzazione errori di connessione.
- Export dello storico dati tramite file CSV.
- Ricevere dati in tempo reale dalla vettura.

4.0.2 Should have

- GUI: identificazione tramite colore dei valori che richiedono l'attenzione dell'operatore.
- Accesso utenti.
- Visualizzazione dati tramite grafico real-time.

4.0.3 Could have

- Possibilità di muovere i componenti grafici all'interno dell'interfaccia.
- Possibilità di cambiare il tema dell'interfaccia.

4.0.4 Would have

- Possibilità di inviare comandi alla monoposto.

Design e Architettura del software

5.0.1 Design

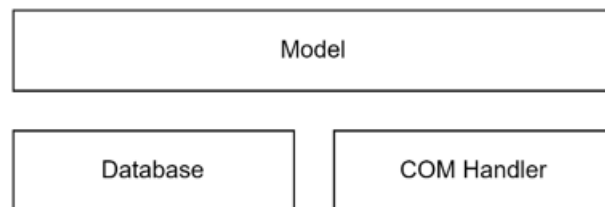
Tra i Design Pattern utilizzati nel progetto si riconoscono:

- **singleton pattern:** Per garantire l'unicità della classe che gestisce il database.
- **Builder pattern:** Abbiamo avuto modo di interagire con questo pattern nella creazione di istanze di oggetti "Gauge".
- **Observable-Observer like pattern:** Utilizzato per evitare un richiamo diretto di una funzione del View su un evento di ricezione dati all'interno del Model.

5.0.2 Stile architetturale

Lo stile architetturale adottato è MVC – Model View Controller per beneficiare del vantaggio di disaccoppiare l'interfaccia grafica dalla logica di programma.

- **View:** gestisce tutta l'interfaccia grafica basata su JavaFX.
- **Model:** gestisce tutte le funzionalità logiche del programma. All'interno del Model è riconoscibile una semplice struttura a strati per cui il model si appoggia sui servizi sottostanti forniti dal gestore del database e dal gestore della porta seriale.
- **Controller:** funge da intermediario tra Model e View: si occupa di richiamare le funzioni del Model corrispondenti ai comandi del View e gestisce l'aggiornamento della GUI con i dati provenienti dal Model.



Ciclo di vita del software

La natura del progetto e la mancanza di esperienza del team nell'ambito dei software telemetrici hanno suggerito l'adozione di un approccio agile.

In generale il metodo utilizzato è SCRUM al quale sono state affiancate tecniche provenienti dall'eXtreme Programming, nello specifico la programmazione in coppia e la proprietà collettiva.

Nelle prime fasi dello sviluppo la prototipazione ha svolto un ruolo fondamentale per due motivi: in primo luogo ha permesso di verificare la fattibilità di alcune soluzioni tecniche alla base del progetto e in secondo luogo ha gettato le fondamenta sulle quali si è evoluto il software.

La scelta di un modello iterativo a cicli brevi ha consentito di adattare i requisiti alle reali esigenze del team durante lo sviluppo e il collaudo del software.

Particolare importanza hanno avuto i test, eseguiti subito dopo l'implementazione di una nuova funzionalità, non solo come strumento di verifica del corretto funzionamento ma anche come strumento di confronto tra le aspettative e il reale comportamento dell'applicazione.

Gestione della configurazione

La gestione della configurazione è stata affidata interamente alla piattaforma GitHub.

La linea guida che il team di sviluppo ha seguito è stata quella di mantenere il main branch sempre funzionante e testato.

Ad esclusione delle modifiche minori per cui è stato eseguito un semplice commit sul main branch per tutte le altre modifiche il processo di sviluppo ha previsto la creazione di un branch secondario, l'implementazione delle modifiche o delle correzioni, la verifica attraverso i test e, in conclusione, il merge con il Main.

In molti casi, per scopi didattici o per semplice curiosità, il team ha creato dei “Dead branch” in cui il team ha sperimentato funzionalità specifiche al fine di valutarne la complessità e l'integrabilità nel progetto.

Gestione del personale

Il team di sviluppo è composto da 2 persone, di pari competenze, senza gerarchie interne.

Ogni membro è responsabile del progetto e si occuperà di tutte le fasi del processo di sviluppo.

I membri del team sono anche parte della squadra che sarà l'utente del sistema. Di conseguenza il team interpreta due ruoli distinti all'interno dell'organizzazione Scrum.

Prima dello sprint, il team si occupa di scrivere e apportare modifiche al backlog adottando il ruolo di Product Owner, dopodiché i membri agiscono separatamente o congiuntamente nella realizzazione degli obiettivi selezionati nello sprint planning.

Indipendentemente dalla modalità di lavoro, i membri sono sempre a stretto contatto e il confronto è continuo.

Diagrammi UML

Diagramma attività

Con questo diagramma vogliamo rappresentare le attività che l'utente e il software devono svolgere per ottenere l'esportazione del file csv contenente i dati presenti in tabella.

Questa attività inizia con il click del pulsante denominato "csv export".

Questo aprirà una finestra nella quale possiamo fare diverse attività:

- inserire il nome del file, attività obbligatoria per il salvataggio
- selezione del percorso per il salvataggio del file, attività opzionale, nel caso in cui non si selezioni manualmente un percorso, il file verrà salvato nella posizione predefinita
- pressione pulsante salva, esporta il file solo se precedentemente è stato inserito un nome per il file.

Non c'è un controllo sui dati, se non sono presenti dati in tabella il file esportato conterrà solo l'elenco di sensori, senza alcun dato associato.

Il diagramma termina con l'esportazione del file.

- annullare l'operazione, tramite il tasto "annulla" o il tasto "X", chiude la finestra senza produrre output, quindi termina il diagramma

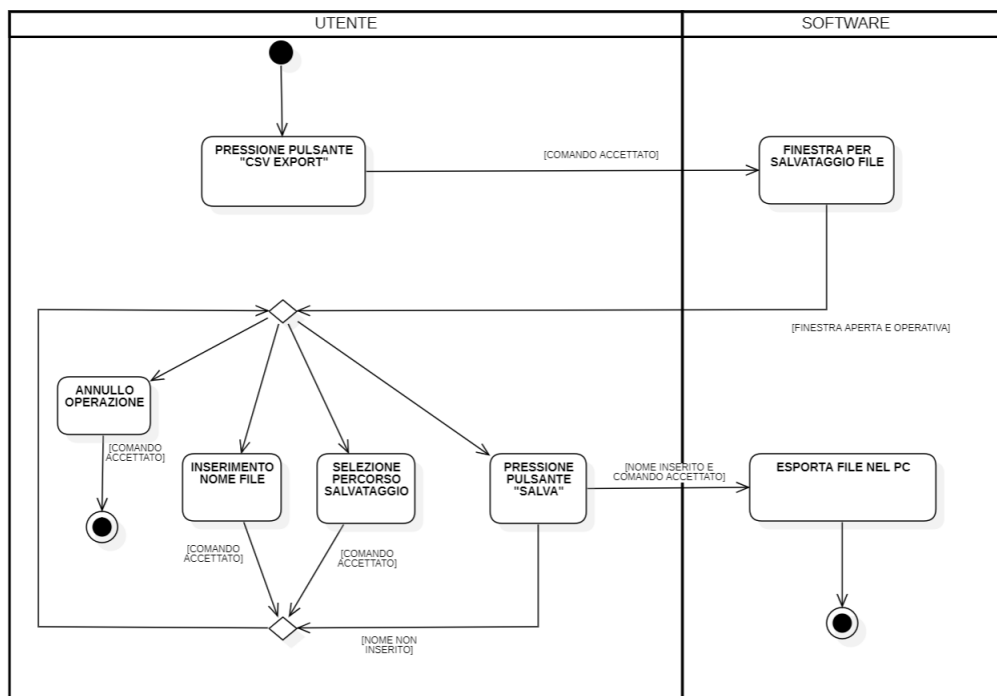


Diagramma casi d'uso

Con il diagramma dei casi d'uso vogliamo descrivere il sistema, ponendo l'attenzione su chi interagisce con esso e cosa può fare ognuno di essi.

Abbiamo tre attori principali che interagiscono con il sistema, sono:

- Microcontrollori: devono poter inviare e ricevere i dati poichè abbiamo un trasmettitore e un ricevitore
- Utente: deve poter richiedere la comunicazione con i microcontrollori, deve poter visualizzare i dati , in real time e nello storico, e deve poter esportare i dati.
- Database: deve gestire i dati nelle operazioni di salvataggio in locale e di esportazione

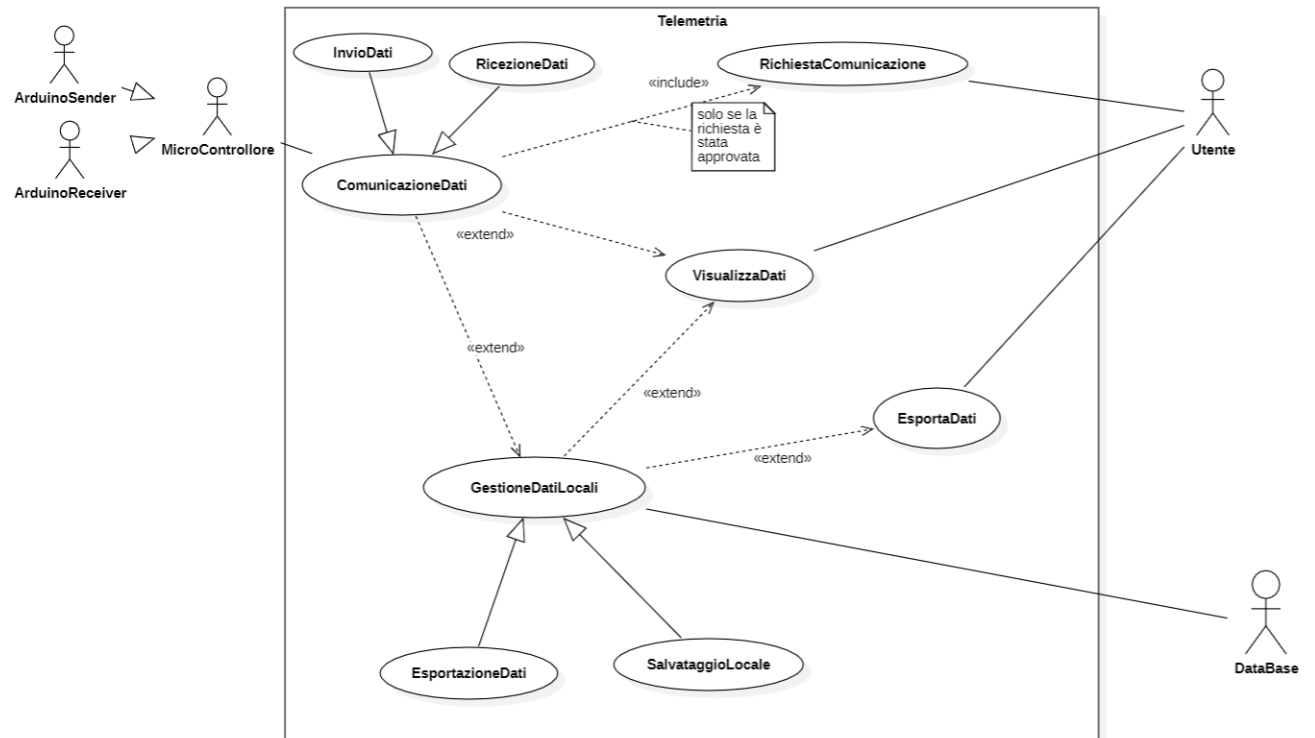


Diagramma di sequenza

Con il seguente diagramma vogliamo rappresentare il processo di scambio dei dati tra il programma e i microcontrollori.

Questo processo si divide in diverse fasi:

- fase 1: dobbiamo verificare che ci siano microcontrollori attivi sulle porte COM, se non ci sono, il processo si interrompe e l'utente dovrà ricominciare dall'inizio
- fase 2: L'utente dovrà selezionare la porta COM desiderata per poter avviare la connessione
- fase 3, Loop di 200ms:
ogni 200ms vengono ricevuti i dati.
si controlla che la connessione con la porta sia ancora attiva (potrebbe esserci un problema con il microcontrollore), se non attiva, termina il processo.
si controlla che l'utente non abbia richiesto la disconnessione della comunicazione con la porta COM, in tal caso termina il processo.

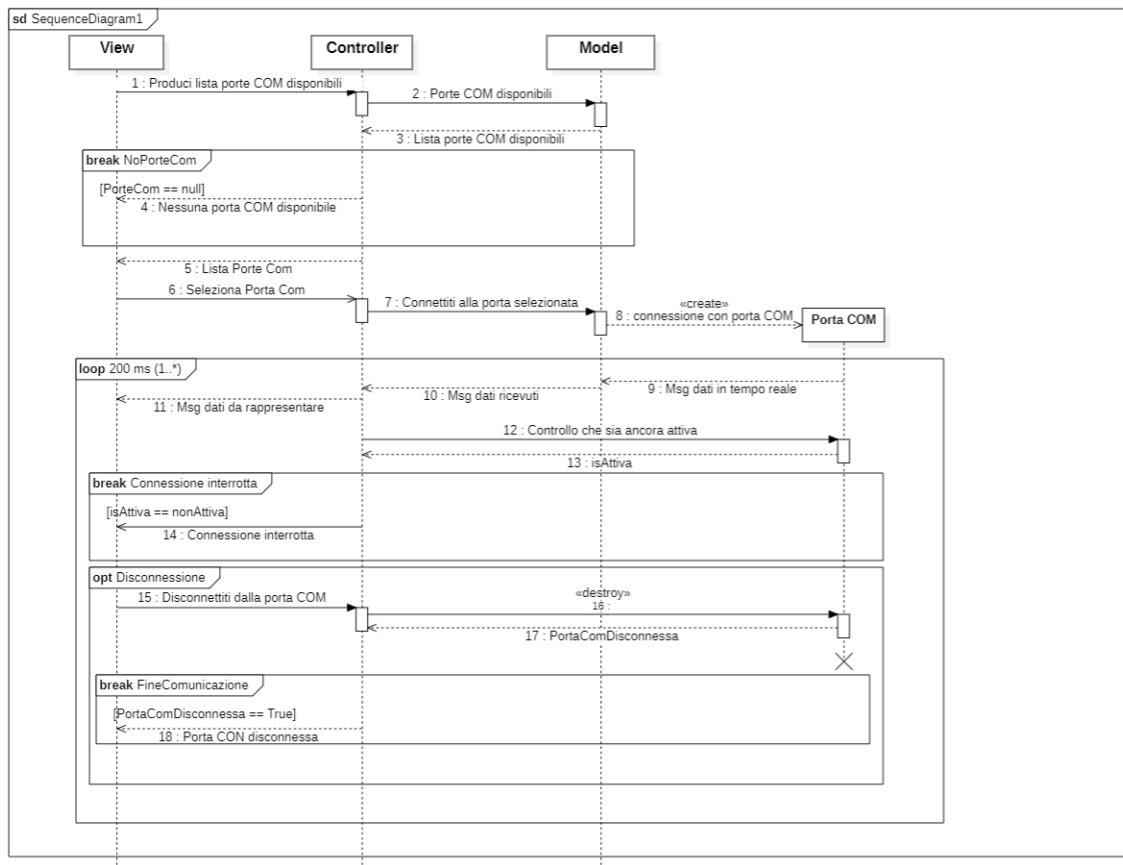


Diagramma della macchina a stati

Con il seguente diagramma vogliamo rappresentare gli stati del sistema durante l'operazione di caricamento dei dati in tabella.

- il primo stato è quello nel quale troviamo l'applicazione aperta nella home page, con la tabella di dati vuota e le date impostate sui valori predefiniti (selezionate automaticamente guardando: **Data inizio** cioè la data attuale del computer e **Data fine** cioè il giorno precedente)

Tramite le operazioni che farà l'utente potremmo trovarci in diversi stati:

- se sceglie la data iniziale tramite l'apposita icona, saremo nello stato nel quale si apre il calendario che permette di selezionare la data iniziale dalla quale prendere i dati.
- se sceglie la data finale tramite l'apposita icona, saremo nello stato nel quale si apre il calendario che permette di selezionare la data finale fino alla quale prendere i dati.
- se seleziona l'icona "load" saremo nello stato nel quale il sistema tenterà di caricare i dati in tabella, quest'ultimo stato porterà il sistema in ulteriori possibili stati:
 - stato nel quale appare un popup a schermo, se le date selezionate non rispettano i vincoli imposti (cioè la data finale deve essere successiva a quella iniziale)
 - primo stato , con date selezionate (se rispettano i vincoli) e tabella vuota (in quelle date non c'erano dati salvati)
 - stato nel quale nella tabella sono stati caricati i dati
- il sistema termina se l'utente seleziona l'icona per la chiusura dell'applicazione

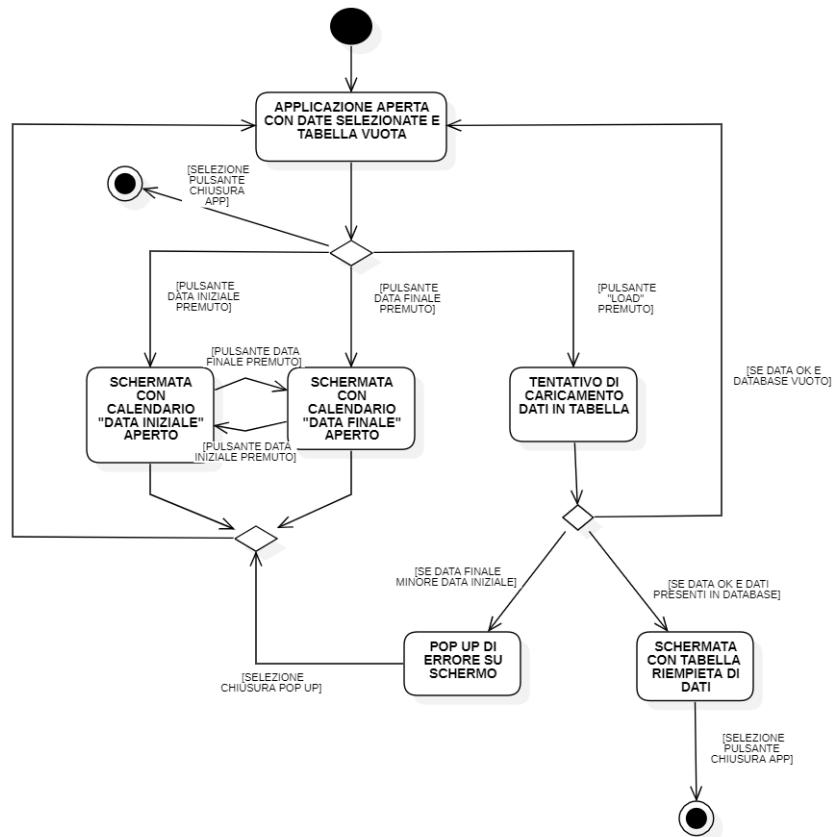


Diagramma delle classi

Nel diagramma delle classi abbiamo rappresentato un pattern MVC (Model-View-Controller) che rappresenta l'architettura di partenza, che utilizziamo per la programmazione del nostro software.

- **view:** nel package sono presenti diverse classi, View è la classe che si occupa dell'interfaccia grafica completa e dell'interazione dell'utente con i componenti grafici del software (GUI). L'interfaccia è definita grazie all'uso di contenitori (classi di javafx chiamate Pane) che ci permettono di comporre il layout tramite contenitori standardizzati. Noi usiamo `BorderPane` per dividere l'interfaccia in 3 sezioni, `TopPane`, `CenterPane` e `BottomPane`, all'interno dei quali inseriamo, sempre tramite `Pane`, elementi grafici disposti secondo le specifiche definite con i clienti.
- **model:** package contenente le classi che gestiscono tutte le funzionalità logiche del programma. All'interno del Model è riconoscibile una semplice struttura a strati per cui il model si appoggia sui servizi sottostanti forniti dal gestore del database e dal gestore della porta seriale.
- **controller:** package che contiene le classi che hanno il compito da fare da intermediario tra Model e View: si occupa di richiamare le funzioni del Model corrispondenti ai comandi del View e gestisce l'aggiornamento della GUI con i dati provenienti dal Model.
- **telemetry:** è il package che contiene la classe principale (Main) che si occupa dell'inizializzazione del software e dell'avvio.

